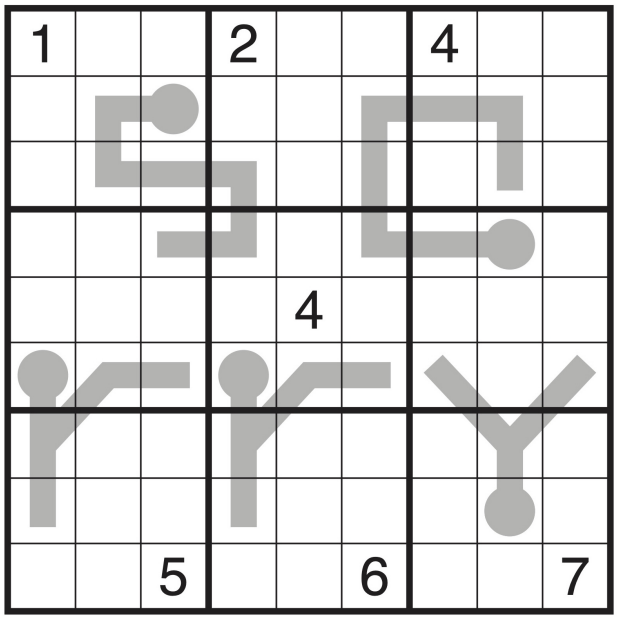
Sudoku Solvers via Combinatorial Optimization

Solving different sudoku variants under different constraints.

Umay Gokturk, Tiffany Gong, Luna Kim



Mathematical Modelling: Discrete Optimization Problems
University of British Columbia
March 2026

Contents

1 Introduction

Sudoku is one of the most well-known combinatorial puzzles. The classic version consists of a (9×9) grid divided into nine (3×3) subgrids. The goal is to fill the grid with the digits 1 through 9 such that each digit appears exactly once in every row, column, and subgrid. Sudoku can be described entirely through logical rules, it can also be formulated as a constraint satisfaction problem.

While the rules of Sudoku are simple, modelling the puzzle mathematically has been widely studied. The classic Sudoku as a linear programming project has already been formulated and solved using linear and integer programming methods. In this paper, we extend the ideas of classic Sudoku by formulating several variants as optimization models. These variants introduce additional constraints that change the structure of the puzzle while also preserving the core Sudoku rules.

2 Problem Statement

We extend the classic Sudoku formulation to explore three variants: Arrow Sudoku, Killer Sudoku, and Thermo Sudoku. Each Sudoku variant is formulated as a feasibility problem with additional constraints on top of all classic Sudoku rules.

3 Classic Sudoku

A Classic Sudoku is a number placement puzzle that has specific constraints to be followed for each placement. A Classic Sudoku consists of (9×9) grid plane, which is divided into 9 (3×3) grids. Each row, column and 3×3 grid should contain only one repetition of each number from 1 to 9.

Objective Function

In linear programming, we normally have an objective function either to maximize or to minimize. But for Sudoku, the objective function does not exist as this is a feasibility problem. If the constraints are satisfied, we can say that the Sudoku problem is solved. Therefore, our dummy objective function is set to 0.

Variables and Parameters

To represent the Sudoku puzzles mathematically, following variables and parameters are used.

- $i, j \in \{1, 2, \ldots, 9\}$: row and column indices.
- $v \in \{1, 2, \ldots, 9\}$: digit value.
- G_{ijv}: binary variable, 1 if the cell (i, j) contains digit value v, 0 otherwise.
- D is the set of known prefilled values

Constraints

To successfully solve the Classic Sudoku, the following constraints should be met.

1. Cell uniqueness: Each cell should contain only 1 value.

$$\sum_{v=1}^{9} G_{ijv} = 1, \qquad \forall i, j \tag{1}$$

```
    for row in rows:
        for col in cols:
            prob.addConstraint(plp.LpConstraint(
                e=plp.lpSum([grid_vars[row][col][value] for value in values]),
                sense=plp.LpConstraintEQ,
                rhs=1,
                name=f"constraint_sum_{row}_{col}"))

```

2. Row uniqueness: For each row, numbers 1 to 9 can appear only once.

$$\sum_{j=1}^{9} G_{ijv} = 1, \qquad \forall i, v \tag{2}$$

```
    for row in rows:
        for value in values:
            prob.addConstraint(plp.LpConstraint(
                e=plp.lpSum([grid_vars[row][col][value]  for col in cols]),
                sense=plp.LpConstraintEQ,
                rhs=1,
                name=f"constraint_uniq_row_{row}_{value}"))

```

3. Column uniqueness: For each column, numbers from 1 to 9 can appear only once.

$$\sum_{i=1}^{9} G_{ijv} = 1, \qquad \forall j, v \tag{3}$$

```
    for col in cols:
        for value in values:
            prob.addConstraint(plp.LpConstraint(
                e=plp.lpSum([grid_vars[row][col][value]  for row in rows]),
                sense=plp.LpConstraintEQ,
                rhs=1,
                name=f"constraint_uniq_col_{col}_{value}"))

```

4. Box uniqueness: For each 3×3 box, numbers from 1 to 9 can appear only once.

$$\sum_{j=3q-2}^{3q} \sum_{i=3p-2}^{3p} G_{ijv} = 1, \quad \forall v, \ \forall p, q \in \{1, 2, 3\} \tag{4}$$

```
    for grid in grids:
        grid_row  = int(grid/3)
        grid_col  = int(grid%3)

        for value in values:
            prob.addConstraint(plp.LpConstraint(
                e=plp.lpSum([grid_vars[grid_row*3+row][grid_col*3+col][value]
                    for col in range(0,3) for row in range(0,3)]),
                sense=plp.LpConstraintEQ,
                rhs=1,
                name=f"constraint_uniq_grid_{grid}_{value}"))

```

5. Pre-filled cells: The pre-filled cells are already known, therefore the decision variable is immediately set to 1:

$$G_{ijv} = 1, \qquad \forall i, j, v \in D \tag{5}$$

where d is the pre-filled value of the cell.

```
    for row in rows:
        for col in cols:
            if(input_sudoku[row][col] != 0):
                prob.addConstraint(plp.LpConstraint(
                    e=plp.lpSum([grid_vars[row][col][value]*value for value in values])
,
                    sense=plp.LpConstraintEQ,
                    rhs=input_sudoku[row][col],
                    name=f"constraint_prefilled_{row}_{col}"))

```

Classic Sudoku Example

```
OPTIMAL LP SOLUTION FOUND
Integer optimization begins...
Long-step dual simplex will be used
+   122: mip =     not found yet >=              -inf        (1; 0)
+   122: >>>>>   0.000000000e+00 >=   0.000000000e+00   0.0% (1; 0)
+   122: mip =   0.000000000e+00 >=     tree is empty   0.0% (0; 1)
INTEGER OPTIMAL SOLUTION FOUND
Time used:   0.0 secs
Memory used: 0.8 Mb (845116 bytes)
Solution Status = Optimal
```

Unsolved

3			8					1
					2			
	4	1	5			8	3	
	2				1			
8	5		4		3		1	7
			7				2	
	8	5			9	7	4	
			1					
9					7			6

Solved

3	6	7	8	9	4	2	5	1
5	9	8	3	1	2	6	7	4
2	4	1	5	7	6	8	3	9
7	2	3	9	8	1	4	6	5
8	5	6	4	2	3	9	1	7
4	1	9	7	6	5	3	2	8
1	8	5	6	3	9	7	4	2
6	7	2	1	4	8	5	9	3
9	3	4	2	5	7	1	8	6

Figure 1: Unsolved and Solved Classic Sudoku with LP

4 Mathematical Tools and Theories

This project is formulated as a Binary Integer Linear Programming (BILP) problem. There is no quantity to minimize or maximize; we are only looking for a feasible solution.

In a BILP, all decision variables are restricted to $\{0, 1\}$. In the Sudoku context, $G_{i,j,v} = 1$ means cell (i, j) is assigned digit v, and 0 otherwise.The constraints that define valid Sudoku solutions: cell uniqueness, row uniqueness, column uniqueness, and box uniqueness, are all linear equalities or inequalities in these binary variables, so the full problem is a valid BILP.

To compute an optimal solution, we use the Python library `PuLP`, which provides tools for defining and solving linear and integer programming problems.

Since Sudoku is a constraint satisfaction problem, the model does not require a meaningful objective function. The objective function is set to a dummy constant of zero, and the solver simply searches for any feasible integer point.

5 Arrow Sudoku

Problem Statement

Arrow Sudoku is a variant of Classic Sudoku, with added arrows that add more constraints. In the Arrow Sudoku, there are arrows on specific cells. The arrows consist of 2 parts; "the base" or "circled" cell that has the sum of the following path of cells on the arrow, and the second part is the "body" or "path" of the arrow whose sum will give the number in the circled cell. This rule becomes the additional constraint in the LP problem of the Classic Sudoku Puzzle.

Variables and Parameters

We use the binary decision variable from the classic formulation:

$$G_{ijv} = \begin{cases} 1 & \text{if cell } (i,j) \text{ contains value } v \\ 0 & \text{otherwise} \end{cases}$$

where $i, j, v \in \{1, \ldots, 9\}$.

Let A be the set of all arrows in the Sudoku and $a \in A$.

- The base of the arrow (containing the sum of the arrow) be denoted by the pair (c_r, c_c) where c_r and c_c represent circle row and column respectively.
- The path of the arrow is denoted by the set $P_a = \{(p_r^1, p_c^1), (p_r^2, p_c^2), ..., (p_r^n, p_c^n)\}$ where p^n denotes each cell on the arrow's path and the pair (p_r^n, p_c^n) denotes the row and column of p^n.

Constraints

In the Arrow Sudoku, the Classic Sudoku constraints still need to be followed. The only difference is an additional constraint that comes from each arrow. On top of the Classic Sudoku constraints we have the following constraint added to the LP problem.

For each arrow, the circle should contains the sum of all the numbers on the arrow's path. So the constraint for each arrow:

$$\sum_{v=1}^{9} v \cdot G_{c_r,\, c_c,\, v} \;=\; \sum_{(p_r^k,\, p_c^k) \in P_a} \sum_{v=1}^{9} v \cdot G_{p_r^k,\, p_c^k,\, v} \qquad \forall\, a \in A \tag{6}$$

Or in standard LP form:

$$\sum_{v=1}^{9} v \cdot G_{c_r,\, c_c,\, v} \;-\; \sum_{(p_r^k,\, p_c^k) \in P_a} \sum_{v=1}^{9} v \cdot G_{p_r^k,\, p_c^k,\, v} \;=\; 0 \qquad \forall\, a \in A \tag{7}$$

```
    for idx, arrow in enumerate(arrows):
        cr, cc = arrow["circle"]
        path = arrow["path"]

        circle_value = plp.lpSum(
            [grid_vars[cr][cc][v] * v for v in values]
```

```
        )

        path_sum = plp.lpSum(
            [grid_vars[pr][pc][v] * v for (pr, pc) in path for v in values]
        )

        prob.addConstraint(
            plp.LpConstraint(
                e=circle_value - path_sum,
                sense=plp.LpConstraintEQ,
                rhs=0,
                name=f"constraint_arrow_{idx}",
            )
        )

```

Results

```
OPTIMAL LP SOLUTION FOUND
Integer optimization begins...
Long-step dual simplex will be used
+   432: mip =     not found yet >=              -inf        (1; 0)
+   432: >>>>>   0.000000000e+00 >=   0.000000000e+00   0.0% (1; 0)
+   432: mip =   0.000000000e+00 >=     tree is empty   0.0% (0; 1)
INTEGER OPTIMAL SOLUTION FOUND
Time used:   0.0 secs
Memory used: 1.2 Mb (1289431 bytes)
Solution Status = Optimal
```

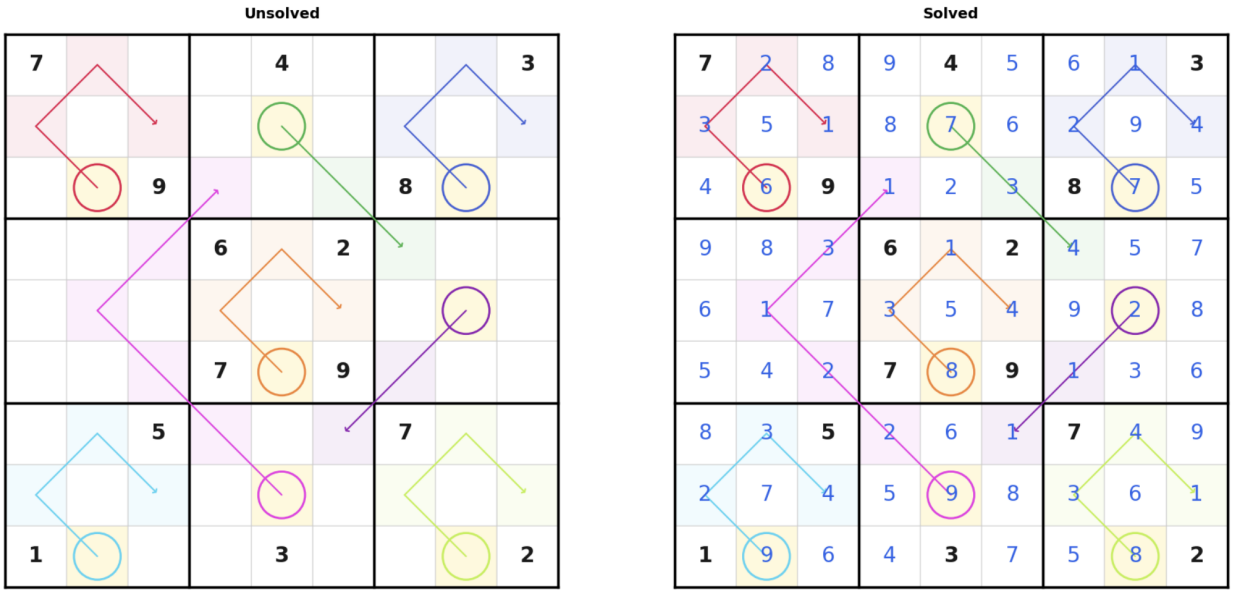


Figure 2: Unsolved and Solved Arrow Sudoku with LP

This example Arrow Sudoku is taken from GM Puzzles website.[4]

Analysis and Discussion

The arrow constraint fits cleanly into the LP formulation, since it is just a linear equality between the weighted sum of binary variables in the circle cell and the weighted sum across the path cells. No auxiliary

variables or linearization techniques were needed to solve the Sudoku. This can be seen from the Solver Log, the LP relaxation is solved first, and the integer optimal solution is found at the root node without any branching since the tree is empty after a single node. This shows that the LP relaxation is tight enough that the branch-and-bound process resolves immediately, which is consistent with Sudoku, since it is a highly constrained LP problem.

From the computational cost perspective; the solve time of under 0.1 seconds confirms that the added constraints do not impact performance compared to Classic Sudoku. For a typical Arrow Sudoku with approximately 10 arrows, the arrow constraints add one equality per arrow, so this does not increase the computational cost much compared to Classic Sudoku constraints.

6 Killer Sudoku

Problem Statement

We extend the classic Sudoku formulation to explore Killer Sudoku, which utilizes cage-sum constraints. Killer Sudoku is a variant of classic Sudoku where the grid is separated into groups of cells called cages. Each cage is labelled with a target sum, and two rules apply to each cage: (i) the digits inside the cage must sum to the target sum and (ii) no digits may repeat within a cage. Typically, Killer Sudoku puzzles have no pre-filled cells; one is generally able to find a solution based on the cage constraints alone.

Variables and Parameters

We use the binary decision variable from the classic formulation:

$$G_{ijv} = \begin{cases} 1 & \text{if cell } (i,j) \text{ contains value } v \\ 0 & \text{otherwise} \end{cases}$$

where $i, j \in \{1, \ldots, 9\}$ and $v \in \{1, \ldots, 9\}$.

The following notation is specific to Killer Sudoku:

- K: number of cages.
- $P_k \subseteq \{1, \ldots, 9\}^2$: set of cells in cage k.
- $T_k \in Z > 0$: target sum for cage k.

Constraints

Constraints 1-5 from the classic Sudoku formulation are still followed in Killer Sudoku. Two new constraints are added to account for the cage rules.

6. Cage sum: The digits placed in cage k must sum to its target T_k.
   For each (i, j):

$$\sum_{(i,j)\in P_k} \sum_{v=1}^{9} v \cdot G_{i,j,v} = T_k \qquad \forall\, k \in \{1, \ldots, K\}. \tag{8}$$

```
    for cage_id, (target, cells) in enumerate(cages):
        prob.addConstraint(plp.LpConstraint(
            e=plp.lpSum([grid_vars[row][col][value] * value for (row, col) in cells for
 value in values]),
            sense=plp.LpConstraintEQ,
            rhs=target,
            name=f"constraint_cage_sum_{cage_id}"))

```

7. Cell uniqueness: No digit may appear more than once within a cage.

   For each (i, j):

$$\sum_{(i,j)\in P_k} G_{i,j,v} \leq 1 \qquad \forall\, v, \forall\, k \in \{1, \ldots, K\} \tag{9}$$

```
    for cage_id, (target, cells) in enumerate(cages):
        for value in values:
            prob.addConstraint(plp.LpConstraint(
                e=plp.lpSum([grid_vars[row][col][value] for (row, col) in cells]),
                sense=plp.LpConstraintLE,
                rhs=1,
                name=f"constraint_cage_uniq_{cage_id}_{value}"))

```

It is worth noting that constraint 7 is not redundant even though constraints 1-4 already prevent digit repetition within every row, column, and box. A cage can span multiple rows and columns, and a digit could legally appear twice in the same cage if we were to refer to only constraints 1-4 alone.

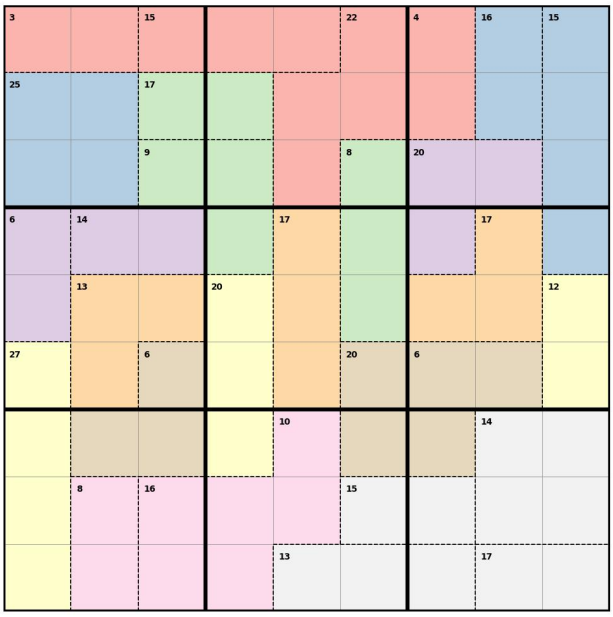
Results

We tested the solver on two instances of varying difficulty. The solver successfully returned a feasible solution that satisfied all Sudoku and cage constraints.

Example 1

The first instance is a standard Killer Sudoku puzzle used to verify that the model correctly enforces cage sums and cage uniqueness constraints. This example is taken from Wikipedia [7].

Problem



Our Result

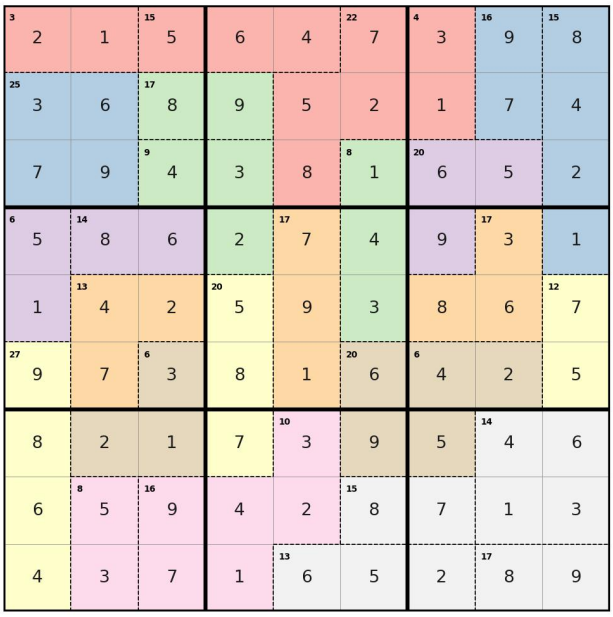


Figure 3: Problem and Our Result of Killer Sudoku Example 1

Example 2

The second example is taken from the *Times* and is rated "Deadly" (Killer Sudoku No. 5849, February 3, 2018)) [6]. This puzzle contains larger cages and more complex cage interactions, making it more challenging to solve by hand.

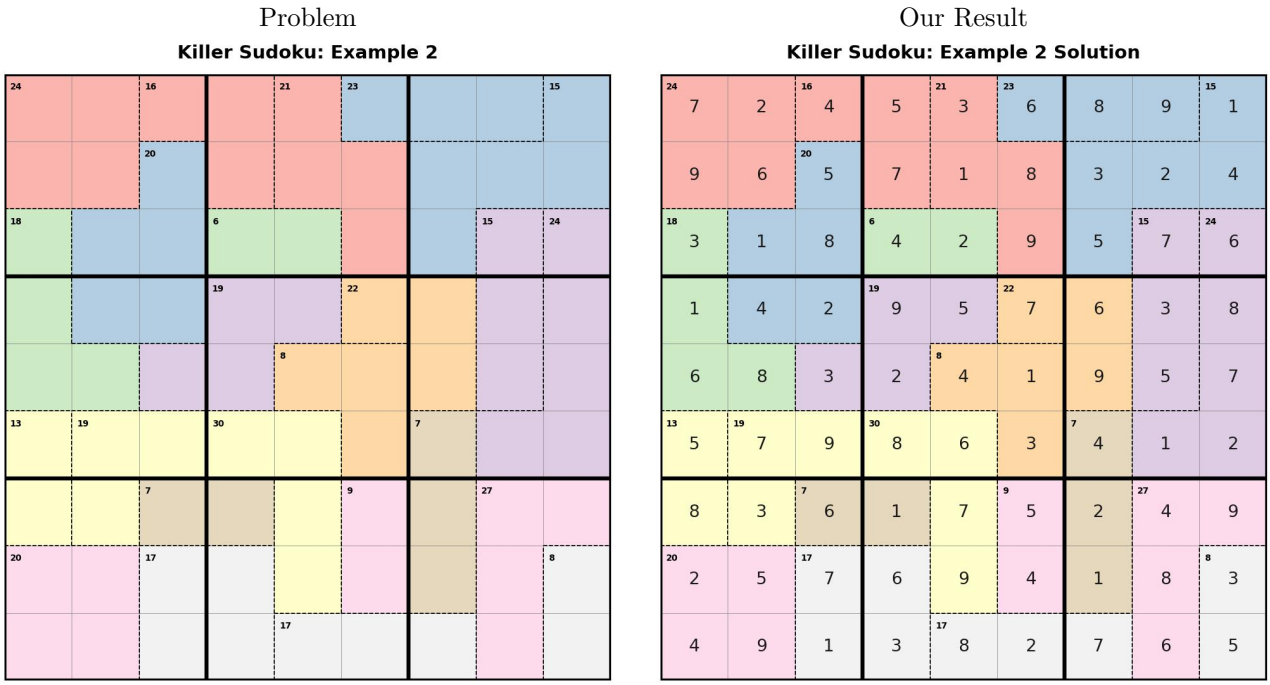


Figure 4: Problem and Our Result of Killer Sudoku Example 2

Runtime Comparison

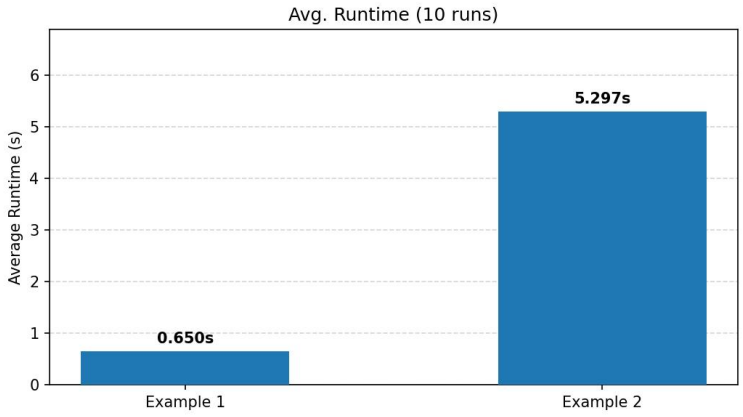


Figure 5: Average Runtime for Killer Sudoku Examples (10 runs per each)

Analysis and Discussion

The solver's runtime was clearly scaled with puzzle difficulty. Example 1 (Figure 3) had an average runtime of 0.650 seconds, while the "Deadly" rated Example 2 (Figure 4) required and average of 5.297 seconds. This increase is likely due to the solver having to explore a deeper search tree driven by the cage constraints

being weaker, since a large target sum can be satisfied by many different digit combinations. These examples contrast heavily with classic Sudoku and arrow Sudoku, which both resolved in 0.0 seconds with likely no branching

The Killer Sudoku formulation adds two constraints on top of the classic model, which is modest in count. However, these constraints couple variables across different rows and columns simultaneously, which is likely what makes the LP harder to solve than in the classic case

The results demonstrate that the BILP formulation is capable of solving any feasible Killer Sudoku puzzle using the mathematical constraints defined in the model. Killer Sudoku has no pre-filled cells and relies entirely on cage sums to drive inference, making it more computationally demanding than some other variants may be.

7 Thermo Sudoku

Problem Statement

Thermo Sudoku extends the classic 9×9 Sudoku formulation by introducing a thermometer-shaped sequence of cells on the grid. Each thermometer starts from a bulb (commonly has a circular image on one cell) and ends at a tip. The values on any thermometer must be strictly increasing from bulb to tip, while all classic Sudoku rules still apply.

Variables and Parameters

We use the binary decision variable from the classic formulation:

$$G_{ijv} = \begin{cases} 1 & \text{if cell } (i,j) \text{ contains value } v \\ 0 & \text{otherwise} \end{cases}$$

where $i, j, v \in \{1, \ldots, 9\}$.

We introduce the following additional notations for the thermo constraints:

- T: the set of all the thermometers in the Sudoku.
- $t \in T$: a single thermometer that is represented as an ordered sequence of cell positions $(r_1, c_1), \ldots, (r_m, c_m)$ from bulb to tip, where $r_n \in \{1, \ldots, 9\},\ c_n \in \{1, \ldots, 9\}$
- m: the number of cells in the thermometer t.

Constraints

The classic Sudoku constraints are inherited without changes from the base formulation [1]. The thermo constraint is as follows.

Let $H_{ij} = \sum_{v=1}^{9} v \cdot G_{ijv}$, where (i, j) are the row and column indices assigned to the cell.
For each thermometer $t = \big((r_1, c_1), \ldots, (r_m, c_m)\big) \in T$, and also for each consecutive pair of cells in t:

$$H_{r_n, c_n} < H_{r_{n+1}, c_{n+1}}, \quad \forall t \in T,\ n = 1, \ldots, (m-1)$$

Since exactly one G_{ijv} equals 1 for each cell, H_{ij} evaluates for the unique value. This enforces strict increase from bulb to tip. The constraint remains linear because H_{ij} is a linear expression of the binary variables G_{ijv}.

The thermo constraint is implemented as follows.

```python
def add_thermo_sudoku_constraints(prob, grid_vars, values, thermometer_list):
    t_idx = 0
    for t in thermometer_list:
        for pos in range(len(t) - 1):
            prev_row, prev_col = t[pos][0] - 1, t[pos][1] - 1
            next_row, next_col = t[pos + 1][0] - 1, t[pos + 1][1] - 1

            prev_val = plp.lpSum([grid_vars[prev_row][prev_col][v] * v for v in values])
            next_val = plp.lpSum([grid_vars[next_row][next_col][v] * v for v in values])

            prob.addConstraint(plp.LpConstraint(e=next_val - prev_val,
                                                sense=plp.LpConstraintGE,
                                                rhs=1,
                                                name=f"Thermo_{t_idx}_{pos}"))
        t_idx += 1
```

Thermo Sudoku Constraints in Python using PuLP

The *rhs* parameter of the class *pulp.LpConstraint* only takes a numerical value, and the parameter *sense* has options of *LpConstraintEQ*, *LpConstraintGE*, or *LpConstraintLE* only. Thus, in the code, the constraint is adjusted to $H_{r_{n+1},c_{n+1}} - H_{r_n,c_n} \geq 1$, which is equivalent to the constraint along with the classic Sudoku constraints.

Results

Three Thermo Sudoku were solved as examples, using the BILP formulation implemented in Python with PuLP. All three puzzles were solved to optimality.

The three Thermo Sudoku puzzles used as examples in this paper are sourced from GM Puzzles [1]: Example 1 by Clover [2], Example 2 by Philip Newman [3], and Example 3 by Thomas Snyder [5]. The 'Our Result' visualizations were generated with the assistance of Claude (Anthropic, 2026). [2]

Example 1

Total 12 thermometers, simply shaped as horizontal lines.

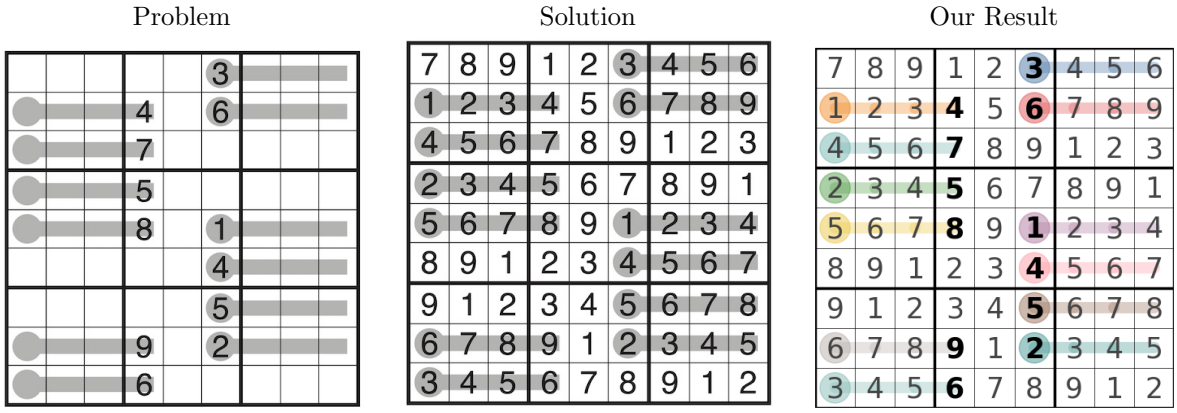


Figure 6: Problem, Reference Solution, and Our Result of Thermo Sudoku Example 1

Example 2

Total 8 thermometers, with zigzag paths.

[1] https://www.gmpuzzles.com
[2] https://claude.ai

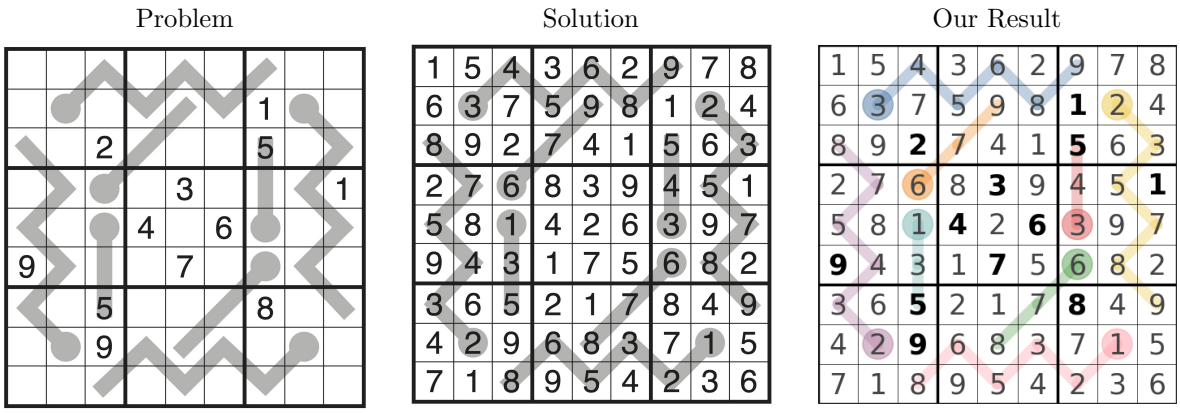


Figure 7: Problem, Reference Solution, and Our Result of Thermo Sudoku Example 2

Example 3

Total 8 thermometers, shaped 'sorry'. The letter 'r' has two different constraints: one is a vertical line path, and another is 'v' like shape.

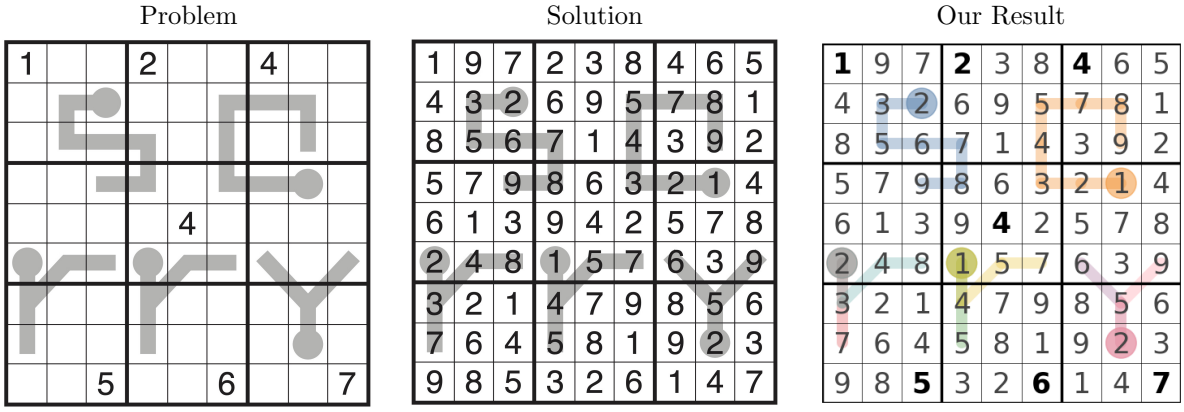


Figure 8: Problem, Reference Solution, and Our Result of Thermo Sudoku Example 3

Runtime Comparison

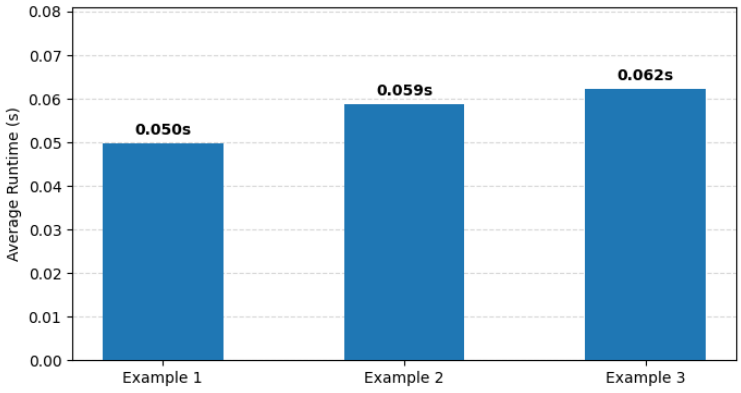


Figure 9: Average Runtime for Thermo Sudoku Examples (10 runs per each)

For quick comparison, classic Sudoku's Average runtime of 10 runs is 0.048s, and 0.054s for Diagonal Sudoku.

Analysis and Discussion

All three puzzles were solved optimally, demonstrating that the thermo constraints are integrated well into the classic Sudoku solver. Compared to classic Sudoku, the thermo Sudoku problem can yield optimal solution with a smaller number of pre-filled clues as the thermo constraints restrict more powerfully.
For example, having a maximum length of thermometer, $m = 9$, can immediately give the values $\{1, \ldots, 9\}$ from bulb cell to tip cell, which tightly restricts the search space and become additional clues to the puzzle.

The thermometers of Example 1 are all horizontal, making the constraints affect rows locally. This result shows horizontal shift-like pattern, where each row has values from 1 to 9 with cyclic shift. Example 2 and 3 are more complex as there is no axis-aligned thermometer path as in Example 1, interacting with the constraints in less predictable ways. However, we found the solver handles all three examples equally well. A key observation is that the total number of constraints grows with the number of thermometers. Each thermometer of length m adds $m-1$ inequality constraints, which makes Thermo Sudoku (computationally) no harder than classic Sudoku in general.

The three Thermo Sudoku examples all solved in under 0.1 seconds on average across 10 runs: Example 1 (0.050s), Example 2 (0.059s), and Example 3 (0.062s). Example 2 and 3 are a bit slower than Example 1, likely due to they are more complex than Example 1, which has simple axis aligned thermometer constraints. Overall, runtime remained consistent across all three, and this confirms that thermo Sudoku constraints do not significantly increase computational cost relative to classic Sudoku.

8 Conclusion

This paper explores three different variants of classic Sudoku: Arrow Sudoku, Killer Sudoku, and Thermo Sudoku. Each Sudoku puzzle is formulated as a BILP, and the variant specific constraints are formulated with linear equality or inequality equations, so they are easily integrated into classic Sudoku formulation.

The test puzzles for all three variants were solved using Python's PuLP library. This means that the additional constraints do not significantly increase the computational complexity and time cost over the classic Sudoku solver. The number of additional constraints grows with the number of arrows, cages, or thermometers, but the problem remains solvable in practice across all variants overall. From this project, we found that all three Sudoku variants can be formulated consistently within the same mathematical framework, demonstrating that BILP is a flexible and powerful tool for constraint-based puzzles.

9 Limitations

While the BILP formulation successfully solves all variants tested, there exists limitations of the model. The model assumes each puzzle has a unique feasible solution; if multiple solutions exist, the solver just returns one silently. Runtime also seems to grow with puzzle complexity, which could be an issue with larger and more complex puzzles. Finally, the solver operates as a black box; it returns a completed grid but provides no human-interpretable reasoning for its process.

10 Future Work

This project focused on modeling several Sudoku variants as a feasibility problem using integer programming. While the model successfully solves the puzzles considered, several possible extensions could be explored in future work.

Combining Variants

Each variant simply adds constraints to the base formulation; thus, in principle, it may be straightforward to combine the rules of various variants. Seeing how the solver handles these hybrid puzzles could be a next step.

Alternative Solving Methods

In this paper, we used binary integer linear programming to solve each puzzle. However, Sudoku can also be solved using other approaches, such as brute-force backtracking algorithms or stochastic search methods. Exploring how these different approaches perform on the different Sudoku variants could provide insight into whether certain solving methods are better suited for particular variants.

Larger Grid Sizes

Another possible extension would be to apply the model to larger Sudoku grids, such as 16×16 or 25×25. This would increase the number of binary variables and constraints, and it would be interesting to see how well the formulation scales.

Algorithm Efficiency

Since this project focused on exploring variants on the 9×9 Sudoku, it does not guarantee efficiency for larger grids or combined variant type of Sudoku. If the puzzle becomes larger, a bottleneck may occur. Future work could investigate more efficient solving strategies, such as logical deduction, pre-filling some cells before passing the puzzle to the solver.

Difficulty Scoring

Difficulty to a human and difficulty for the BILP solver are not the same thing. However, the project could be extended to include a numeric difficulty score. Solver runtime could serve as an indicator for difficulty, as seen in the Killer Sudoku results; harder-rated puzzles took longer to solve. Combined with many examples of human-rated puzzles, this could help build a more meaningful difficulty metric.

Generating Puzzles

This project focused solely on solving given Sudoku puzzles. Investigating puzzle generation could be an interesting extension. It could involve placing arrows or thermometers on the grid, and control difficulty level with pre-filled clues so that the resulting puzzle yields a valid puzzle with a unique solution.

11 Code

GitHub Repository: `https://github.com/lunakim-ubc/sudoku`

References

[1] Fahren Bukhari, Sri Nurdiati, Mohamad K. Najib, and Nandika Safiqri. Formulation of sudoku puzzle using binary integer linear programming and its implementation in julia, python, and minizinc. *Jambura journal of mathematics*, 4(2), 2022.

[2] Clover. Thermo sudoku example 1, 2025.

[3] Philip Newman. Thermo sudoku example 2, 2025.

[4] Thomas Snyder. Arrow sudoku rules and info, 2013. Published January 22, 2013. Grandmaster Puzzles (The Art of Puzzles).

[5] Thomas Snyder. Thermo sudoku example 3, 2025.

[6] The Times. Killer sudoku no. 5849, 2018.

[7] Wikipedia contributors. Killer sudoku, 2026.